

Natural Language Processing

TP 9 – Next Word Prediction using RNN

Chapter 7 – Deep Learning for NLP

Dr BENDIABDALLAH M.H

Université Belhadj BOUCHAIB - Ain Temouchent

Practical Session Overview

In this practical session, students implement a next word prediction system using a recurrent neural network. The work is based on a CSV corpus of sentences about country capitals. The practical session covers the complete pipeline: dataset loading, text preprocessing, vocabulary construction, training example generation, padding of variable-length sequences, definition of an RNN model, training, evaluation, plotting learning curves and predicting the next word for several input sequences.

Learning Objectives

At the end of this practical session, students should be able to:

- understand the idea of **language modelling**;
- explain how **temporal dependencies** appear in text sequences;
- explain the role of a simple **RNN** for sequence modelling;
- prepare a text corpus for next word prediction;
- train and evaluate recurrent neural models in PyTorch;
- interpret the predicted next words and compare model behaviour.

Dataset Used in This TP

This practical session uses the file `capital.csv` and its `sentence` column as the text corpus. Each row contains a sentence related to world capitals. The loaded CSV sentences are then stored in the Python list `corpus`.

1. Step 1 - Import the Libraries

The practical session begins by importing standard Python utilities, PyTorch modules and plotting tools. It also fixes the random seed for reproducibility and chooses the available device (CPU or GPU).

```
import math
import random
import re
from collections import Counter

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

import matplotlib.pyplot as plt
```

```
SEED = 42
random.seed(SEED)
torch.manual_seed(SEED)
```

2. Step 2 - Load the Corpus from CSV

The file is loaded using Pandas. The code reads the `sentence` column and converts it into a Python list called `corpus`.

```
import pandas as pd

# Read CSV file using the sentence column
df = pd.read_csv('capital.csv')
corpus = df['sentence'].tolist()
```

3. Step 3 - Preprocess and Tokenise the Text

The next step lowercases the text, removes non-letter characters and splits the cleaned sentence into tokens. The preprocessing function used in this TP is the following.

```
def tokenize(text):
    text = text.lower().strip()
    text = re.sub(r"[^a-z\s]", "", text)
    return text.split()

tokenized_corpus = [tokenize(sentence) for sentence in corpus]
```

Example:

Original sentence	Tokenised sentence
The capital of France is Paris.	['the', 'capital', 'of', 'france', 'is', 'paris']

4. Step 4 - Build the Vocabulary

The vocabulary is built from all words in the tokenised corpus. Two special tokens are inserted: `<PAD>` for padding and `<UNK>` for unknown words. The code then builds both mappings: `word2idx` and `idx2word`.

```
special_tokens = ["<PAD>", "<UNK>"]

all_words = [word for sentence in tokenized_corpus for word in sentence]
word_counts = Counter(all_words)

vocab = special_tokens + sorted(word_counts.keys())
word2idx = {word: idx for idx, word in enumerate(vocab)}
idx2word = {idx: word for word, idx in word2idx.items()}

vocab_size = len(vocab)
```

Key idea: each word is converted into an integer. Neural networks cannot directly process raw strings, so words must be mapped to numerical indices.

5. Step 5 - Create Next-Word Training Examples

The model must learn from many **input sequence** -> **target word** pairs. For each tokenised sentence, the code gradually increases the input sequence and uses the next token as the target.

```
def encode_tokens(tokens, word2idx):
    return [word2idx.get(tok, word2idx["<UNK>"]) for tok in tokens]

examples = []

for sentence_tokens in tokenized_corpus:
    encoded = encode_tokens(sentence_tokens, word2idx)
    for i in range(1, len(encoded)):
        input_seq = encoded[:i]
        target = encoded[i]
        examples.append((input_seq, target))
```

Illustration with one sentence:

For the sentence **the capital of france is paris**, the generated examples are:

Input sequence	Target
the	capital
the capital	of
the capital of	france
the capital of france	is
the capital of france is	paris

6. Step 6 - Split into Training and Test Sets

Once all examples are created, the code shuffles them and uses a 90% / 10% split. This allows the models to be trained on one subset and evaluated on unseen examples from the same corpus.

```
random.shuffle(examples)
split_idx = int(0.9 * len(examples))

train_examples = examples[:split_idx]
test_examples = examples[split_idx:]
```

7. Step 7 - Create the PyTorch Dataset and Padding Function

Input sequences do not all have the same length. For example, **['the']** and **['the', 'capital', 'of', 'france']** have different lengths. Neural networks process mini-batches more easily when sequences in the same batch share the same length, so the code pads shorter sequences with the token **<PAD>**. This is implemented using a custom dataset class and a custom **collate_fn**.

```
class NextWordDataset(Dataset):
    def __init__(self, examples):
        self.examples = examples

    def __len__(self):
        return len(self.examples)

    def __getitem__(self, idx):
        return self.examples[idx]
```

```

def collate_fn(batch):
    sequences = [torch.tensor(item[0], dtype=torch.long) for item in batch]
    targets = torch.tensor([item[1] for item in batch], dtype=torch.long)
    lengths = torch.tensor([len(seq) for seq in sequences], dtype=torch.long)

    padded_sequences = nn.utils.rnn.pad_sequence(
        sequences,
        batch_first=True,
        padding_value=word2idx["<PAD>"]
    )

    return padded_sequences, lengths, targets

batch_size = 64

train_dataset = NextWordDataset(train_examples)
test_dataset = NextWordDataset(test_examples)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, collate_fn=
    collate_fn)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, collate_fn=
    collate_fn)

sample_batch = next(iter(train_loader))
for item in sample_batch:
    print(item.shape if hasattr(item, "shape") else item)

```

Returned elements in each batch:

- `padded_sequences`: the input sequences padded to the same length;
- `lengths`: the real lengths before padding;
- `targets`: the next words to predict.

8. Step 8 - Define the Generic Recurrent Model

The code defines a class called `NextWordPredictor`. It transforms word indices into embeddings, processes the sequence with an RNN layer, then uses the final hidden representation to predict the next word.

```

class NextWordPredictor(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, dropout=0.2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=word2idx["<
PAD>"])
        self.rnn = nn.RNN(embedding_dim, hidden_dim, batch_first=True, dropout=dropout if
dropout > 0 else 0)

        self.fc = nn.Linear(hidden_dim, vocab_size)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, lengths):
        embedded = self.dropout(self.embedding(x))

        packed = nn.utils.rnn.pack_padded_sequence(

```

```

        embedded, lengths.cpu(), batch_first=True, enforce_sorted=False
    )

    _, hidden = self.rnn(packed)

    # Use the last hidden state
    final_hidden = self.dropout(hidden[-1])
    logits = self.fc(final_hidden)
    return logits

```

Important points:

- the embedding layer converts token indices into dense vectors;
- `pack_padded_sequence` avoids wasting computation on padding tokens;
- the RNN returns a hidden state that summarises the sequence context;
- the final linear layer projects the hidden state to the whole vocabulary size.

9. Step 9 - Define the Training and Evaluation Functions

The code defines two functions: one for computing accuracy from logits and one for performing the full training and evaluation loop. During each epoch, the model is trained on the training loader and then evaluated on the test loader. The history of losses and accuracies is stored for later plotting.

```

def accuracy_from_logits(logits, targets):
    preds = torch.argmax(logits, dim=1)
    correct = (preds == targets).sum().item()
    total = targets.size(0)
    return correct / total

def train_model(model, train_loader, test_loader, epochs=20, lr=0.001):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    history = {
        "train_loss": [],
        "train_acc": [],
        "test_loss": [],
        "test_acc": []
    }

    for epoch in range(epochs):
        model.train()
        total_train_loss = 0.0
        total_train_acc = 0.0
        n_train_batches = 0

        for x, lengths, y in train_loader:
            x, lengths, y = x.to(device), lengths.to(device), y.to(device)

            optimizer.zero_grad()
            logits = model(x, lengths)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()

```

```

        total_train_loss += loss.item()
        total_train_acc += accuracy_from_logits(logits, y)
        n_train_batches += 1

    train_loss = total_train_loss / n_train_batches
    train_acc = total_train_acc / n_train_batches

    model.eval()
    total_test_loss = 0.0
    total_test_acc = 0.0
    n_test_batches = 0

    with torch.no_grad():
        for x, lengths, y in test_loader:
            x, lengths, y = x.to(device), lengths.to(device), y.to(device)
            logits = model(x, lengths)
            loss = criterion(logits, y)

            total_test_loss += loss.item()
            total_test_acc += accuracy_from_logits(logits, y)
            n_test_batches += 1

    test_loss = total_test_loss / n_test_batches
    test_acc = total_test_acc / n_test_batches

    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["test_loss"].append(test_loss)
    history["test_acc"].append(test_acc)

    print(
        f"Epoch {epoch+1:02d}/{epochs} | "
        f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
        f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.4f}"
    )

    return history

```

10. Step 10 - Set the Hyperparameters

This practical session uses the following values for the embedding dimension, hidden size, number of epochs and learning rate.

```

embedding_dim = 128
hidden_dim = 256
epochs = 5
lr = 0.001

```

11. Step 11 - Train the RNN Model

This experiment trains a simple recurrent neural network.

```

rnn_model = NextWordPredictor(vocab_size, embedding_dim, hidden_dim)
rnn_history = train_model(rnn_model, train_loader, test_loader, epochs=epochs, lr=lr)

```

12. Step 12 - Plot the Learning Curves

After training the RNN model, the code plots training loss, test loss, training accuracy and test accuracy to observe convergence behaviour.

```
#
```

13. Step 13 - Predict the Next Word

The final practical function predicts the next word for an input phrase. It tokenises the input, encodes it, sends it through the selected model and returns the **top-k** most probable words with their probabilities. The code then tests several example phrases.

```
def predict_next_word(model, text, top_k=5):
    model.eval()
    tokens = tokenize(text)
    encoded = [word2idx.get(tok, word2idx["<UNK>"]) for tok in tokens]

    x = torch.tensor([encoded], dtype=torch.long).to(device)
    lengths = torch.tensor([len(encoded)], dtype=torch.long).to(device)

    with torch.no_grad():
        logits = model(x, lengths)
        probs = torch.softmax(logits, dim=1)
        top_probs, top_indices = torch.topk(probs, k=top_k, dim=1)

    results = []
    for prob, idx in zip(top_probs[0].cpu().tolist(), top_indices[0].cpu().tolist()):
        results.append((idx2word[idx], prob))
    return results

test_phrases = [
    "capital of algeria is",
    "capital of france is",
]

for phrase in test_phrases:
    print(f"\nInput: '{phrase}'")
    print("RNN:", predict_next_word(rnn_model, phrase))
```

Typical interpretation:

- for capital of france is, the expected next word should be close to paris;
- for capital of algeria is, the expected next word should be close to algiers;
- the model may return several plausible top candidates, especially if the corpus contains multiple sentence forms.